

Nombre (s) : Darío Alberto Sánchez Perzabal Santiago Patricio Gómez Ochoa Ximena Ortiz Gómez María José Croche Álvarez		Matrícula (s): A01738266 A01735171 A01735100 A01734561
Nombre del curso: Implementación de Robótica Inteligente	Nombre del profesor (es): Juan Manuel Ahuactzin Larios Rigoberto Cerino Jiménez Alfredo García Suárez Cesar Torres Huitzil	
Módulo:	Actividad: Reto Semanal 2	
Fecha: 13 de abril del 2026		

Resumen

En este reto semanal se desarrolló un sistema de control en lazo abierto para un robot móvil Puzzlebot utilizando ROS 2, con el objetivo de generar y ejecutar trayectorias definidas por el usuario. El sistema se dividió en dos partes: la primera consistió en el seguimiento de un cuadrado de 2x2 metros mediante una máquina de estados de tres estados (avanzar, pausar y rotar 90°), donde fue necesario realizar un proceso de calibración experimental para compensar las no linealidades de los motores, obteniendo factores de corrección de 0.95 para la distancia y 0.895 para el ángulo. La segunda parte implementó un sistema flexible compuesto por dos nodos, un generador de trayectoria y un controlador, que permite al usuario definir múltiples puntos de destino en el plano junto con las velocidades deseadas (modo 0) o el tiempo de llegada (modo 1). Para la comunicación entre nodos se diseñó un mensaje personalizado llamado MakbetsPose que contiene las coordenadas, velocidades y tiempos de la trayectoria. El controlador utiliza una máquina de estados de cinco estados que calcula las distancias y ángulos relativos entre puntos consecutivos, normaliza los giros al intervalo $[-\pi, \pi]$ y aplica saturación automática cuando las velocidades exceden los límites físicos del robot. Los resultados obtenidos demuestran que el robot ejecuta correctamente trayectorias con coordenadas positivas y negativas en ambos modos de operación, y que los

mecanismos de robustez implementados permiten manejar adecuadamente entradas que superan las capacidades físicas del Puzzlebot.

Objetivos

El objetivo de este reto semanal es desarrollar un nodo en ROS 2 capaz de controlar un robot móvil llamado Puzzlebot mediante un esquema de control de lazo abierto que permite la generación y seguimiento de trayectorias definidas por el usuario considerando la dinámica del sistema. De igual manera entre los objetivos más específicos del reto se encuentra implementar un nodo en ROS que publique comandos de velocidad (`/cmd_vel`) para ejecutar trayectorias definidas como por ejemplo un cuadrado o una figura irregular. Asimismo, este reto tiene como objetivo calcular las velocidades lineales y angulares, o el tiempo requerido para llegar a cierto punto según lo que el usuario defina como entrada, lo que permitirá que el sistema sea flexible ante diferentes situaciones de operación.

Consecuentemente, esta actividad tiene como objetivo analizar el efecto de las perturbaciones, ruido y no linealidades en un sistema de control de lazo abierto y con base en ello implementar un controlador robusto que mejore la precisión del seguimiento de trayectorias. Finalmente, el reto busca integrar el uso de mensajes personalizados tipo Pose y mensajes tipo Twist dentro del entorno de ROS para la comunicación entre nodos y validar el funcionamiento del sistema en el robot real.

Introducción

Para este reto semanal se ha utilizado el framework ROS, el cual permite desarrollar sistemas modulares mediante la comunicación entre nodos a través de tópicos y mensajes. En este contexto, este reto se enfoca en la implementación de un sistema de control en lazo abierto para un robot Puzzlebot con el fin de generar trayectorias y por ende realizar su seguimiento con el robot.

Tomando en cuenta lo anteriormente mencionado, para la implementación de este reto se utilizaron mensajes tipo Pose. Los mensajes tipo Pose son estructuras de datos dentro del paquete *geometry_msgs* utilizadas para representar la posición y orientación del robot en el espacio. Estos mensajes permiten coordinar el movimiento del robot dentro de un entorno físico o simulado. La estructura del mensaje tipo Pose consta de dos principales componentes los cuales son la posición otorgada por un punto x,y,z y la orientación otorgada por un cuaternión (x,y,z,ω) que representa la rotación del objeto [1].

De igual manera, en este reto se ocupan los mensajes tipo Twist los cuales son un tipo de mensaje estándar de ROS que igual son parte del paquete *geometry_msgs* los cuales son

utilizados para controlar el movimiento del robot a través de la velocidad lineal en m/s y la velocidad angular en rad/s. Estos mensajes son sumamente utilizados puesto que especifican la velocidad en un espacio determinado a través de estos dos vectores enviados al tópico */cmd_vel* [2].

Consecuentemente, en este reto se tuvo que implementar un control en lazo abierto de un robot móvil, por lo que de acuerdo con Christ & Wernli (2014), un sistema de control en lazo abierto es un sistema en funcionamiento donde la máquina de estados tiene dos estados básicos que son encendido y apagado. Además, el término lazo abierto se refiere a la falta de retroalimentación de sensores para controlar el funcionamiento de la máquina. Ejemplificando lo anterior, un interruptor de luz es un dispositivo con retroalimentación de lazo abierto dado que al activarse siempre permanece en el estado encendido o apagado hasta que se cambie manualmente [3]. Debido a esto, en un control de lazo abierto el robot depende fuertemente de la precisión de modelo y de las condiciones del entorno, lo que hace que factores como el ruido, las perturbaciones y las no linealidades de los motores puedan afectar la ejecución de las tareas del robot.

En este contexto, el cálculo de la distancia y el ángulo recorrido por el robot móvil es fundamental para la correcta generación de trayectorias. Para ello, se considera la relación geométrica entre puntos consecutivos en el plano, donde la distancia se puede obtener a partir de las diferencias en las coordenadas del robot, mientras que el ángulo de orientación se determina mediante la tangente inversa de las diferencias en las distancias de los dos puntos ingresados por el usuario. A partir de estos valores es posible definir desplazamientos lineales y rotacionales que permiten al Puzzlebot seguir trayectorias específicas. Finalmente, este reto semanal permite comprender la relación entre la cinemática del robot móvil y su control incluyendo el cálculo de distancias y ángulos recorridos a partir del tiempo o las velocidades, lo que permite sentar bases para el desarrollo de sistemas robóticos más complejos.

Solución del problema

Para dar solución a los objetivos del reto semanal 2, primero fue necesario comprender el funcionamiento de las máquinas de estado, pues fueron la base para ambas partes del reto, al igual que la implementación del controlador de lazo abierto, donde fue decisión del equipo que estrategias utilizar para que fuera suficientemente robusto y poder tener en consideración el ruido o perturbaciones que pudieran presentarse. En esta ocasión, el reto se dividió en dos partes, siendo la primera el seguimiento de un cuadrado de 2x2 metros

y la segunda el seguimiento de puntos, por medio de coordenadas de un plano, introducidos por el usuario, donde además define las velocidades lineales y angulares para cada uno de ellos y el robot calcula el tiempo, o caso contrario, introduce los puntos de coordenadas y el tiempo en que debe llegar y el robot calcula las velocidades a las que debe ir.

Parte 1

Para la primera parte del reto, fue necesario crear un nodo *controller*, donde se creó un *publisher* para el nodo */cmd_vel* de mensaje tipo *Twist*, el cual es el encargado de publicar tanto la velocidad lineal como la velocidad angular en tiempo real. Igualmente, en este nodo se creó la máquina de estados basada en tiempo para que el robot siga la trayectoria del cuadrado. La máquina de estados cuenta con 3 estados, mover hacia adelante, parar y rotar 90° . Esto puede observarse en la imagen 1, donde puede apreciarse la máquina de estados de una manera visual.

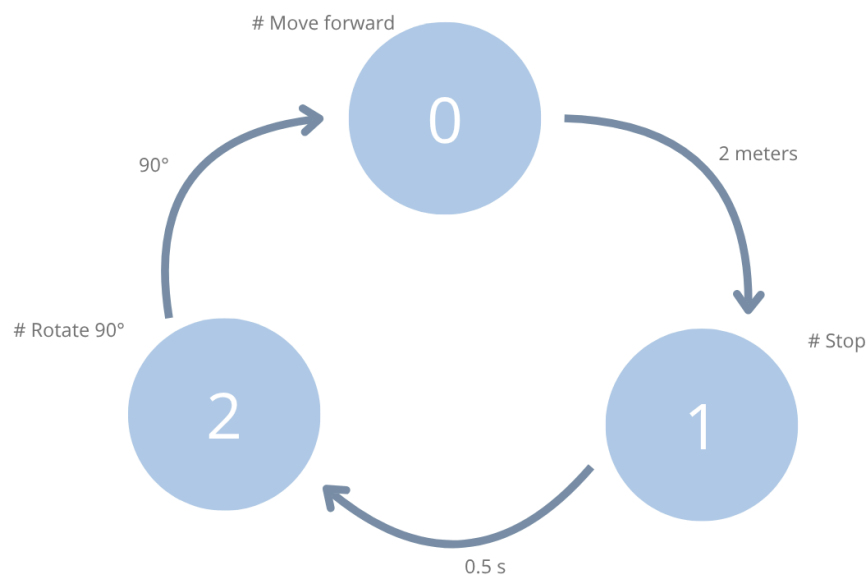


Imagen 1. Diagrama visual de la máquina de estados.

Para el estado 0, es decir el movimiento hacia adelante, el robot se desplaza en línea recta con una velocidad lineal constante y una velocidad angular de 0 rad/s. Una vez que alcanza los 2m, los cuales son medidos gracias a los encoders de los motores en cada rueda, el estado de la máquina de estados cambia al siguiente, donde el estado 1 representa un alto para el robot, siendo entonces las velocidades lineal y angular 0, esto se hace en forma de transición para el siguiente movimiento y evitar comportamientos no deseados. En este estado espera 0.5 s y cambia de estado. El estado 2, es decir la rotación de 90° , se encarga de

girar el robot sobre su propio eje, esto quiere decir que únicamente existe velocidad angular constante y la velocidad lineal es 0 m/s.

Al tratarse de un controlador de lazo abierto, no hay retroalimentación de los sensores para corregir su movimiento, por lo tanto la máquina de estados no cambia de estado basada en mediciones, sino que lo hace por el tiempo de tarda en realizar la acción o movimiento de cada estado ya mencionado, esto se hace independientemente de si el movimiento se hizo o no con la precisión esperada. Es por ello que también fue necesario calibrar manualmente tanto la distancia como el ángulo que gira, pues esto puede variar dependiendo de las condiciones del ambiente en el que se encuentre el robot, pues puede haber más o menos fricción. En nuestro caso fue necesario calibrar la distancia multiplicando los 2m por 0.95, y el ángulo de 90° o $\pi/2$ por 0.895. Una vez haciendo esto, físicamente el robot sí realiza el giro de 90° y recorre una distancia de 2m.

Además para garantizar que el robot ejecutara correctamente la trayectoria deseada, se realizó un proceso de calibración experimental tanto para el desplazamiento lineal como para el movimiento angular. Para ello se llevó a cabo diversas pruebas variando K_p aplicado a la distancia y al ángulo, comparando los valores esperados con los reales. A continuación se muestran los resultados de las calibraciones tanto del movimiento angular como lineal:

Pruebas Ángulos Puzzlebot			
Prueba	Valor K_p	Ángulo teórico (grados)	Ángulo real aproximado (grados)
1	1	90	110
2	0.8	72	75
3	0.85	76.5	80
4	0.87	78.3	85
5	0.88	79.2	85
6	0.885	79.65	85
7	0.89	80.1	87
8	0.895	80.55	90

Imagen 2. Resultados de la calibración del movimiento angular del Puzzlebot

Pruebas Distancia Puzzlebot			
Prueba	Valor K_p	Distancia teórica (metros)	Distancia real aproximada (metros)
1	1	2	2.2
2	0.8	1.6	1.9
3	0.85	1.7	1.97
4	0.9	1.8	1.95
5	0.95	1.9	2

Imagen 3. Resultados de la calibración del movimiento lineal del Puzzlebot

Como se observa en las pruebas realizadas, inicialmente el robot presentaba sobrepasos tanto en distancia como en el ángulo, por ejemplo, para un giro de 90° , el robot alcanzaba aproximadamente 110° , y para una distancia teórica de 2m, recorría alrededor de 2.2 m. A partir de estas mediciones, se ajustaron progresivamente los valores hasta encontrar un factor de corrección adecuado. En el caso del ángulo, se determinó un valor de calibración de aproximadamente 0.895, mientras que para la distancia se utilizó un factor cercano a 0.95, logrando que el robot alcanzara valores reales muy cercanos a los teóricos, como un giro de 90° y un desplazamiento de 2 m. Estos factores de calibración fueron posteriormente utilizados en la segunda parte del reto, permitiendo mejorar la precisión del seguimiento de trayectorias definidas por múltiples puntos del usuario.

Parte 2

Para la segunda parte del reto se desarrolló una estrategia de navegación en la que el usuario introduce una serie de coordenadas en el plano y las velocidades o el tiempo y el robot sigue la trayectoria resultante mediante un control en lazo abierto. A diferencia de la parte 1 de este reto donde la trayectoria ya estaba definida como un cuadrado de 2x2, en esta sección se buscó que el sistema fuera flexible, permitiendo al usuario definir distintos puntos de destino y que además seleccionara la forma en la que deseaba ejecutar el recorrido. Para lograrlo se implementaron 2 nodos en ROS: un nodo generador de trayectoria llamado *path_generator* y un nodo controlador llamado *controller*. El primero se encarga de recibir los datos del usuario y publicarlos en el tópico */pose*, mientras que el segundo procesa dicha información, calcula las variables necesarias para el cálculo del movimiento y publica los comandos de velocidad al robot mediante el tópico */cmd_vel* mediante mensajes tipo Twist

Consecuentemente, se utilizó un mensaje tipo pose personalizado llamado MakbetsPose, el cual funciona como un mensaje tipo pose pero adaptado a las necesidades del reto. Este mensaje contiene arreglos con las coordenadas x y y las cuales son ingresadas mediante un archivo de parámetros (.yaml) y almacenan los puntos que debe de seguir o alcanzar el robot. También, este mensaje personalizado contiene arreglos para las velocidades lineales y velocidades angulares los cuales se utilizan cuando el usuario proporciona directamente las velocidades. El arreglo de tiempo de recorrido se usa cuando el usuario desea que el robot llegue a cada punto en un tiempo dado y la variable *mode* le permite al usuario definir cual de estas dos formas de operación se estará utilizando. La generación de la interfaz del mensaje se declaró en un paquete llamado makbets_pose mediante CMakeLists.txt, donde se incluyó el archivo MakbetsPose.msg para que pudiera ser utilizado

por ambos nodos del sistema. A continuación se muestra la estructura del mensaje personalizado:

```
float64[] x
float64[] y
float64[] linear_speed
float64[] angular_speed
float64[] path_time
int32 mode
```

Posteriormente, se desarrolló el nodo `path_generator`, cuya función principal es solicitar los parámetros del usuario y publicarlos en el tópico `/pose`. El primer parámetro solicitado es el modo de ejecución. Si el usuario selecciona el modo 0, el sistema pide las coordenadas `x` y `y` de cada punto, así como la velocidad lineal y la velocidad angular deseada en cada punto. Si se selecciona el modo 1, el sistema nuevamente pide las coordenadas y el tiempo en el que se desea alcanzar cada uno de los puntos. Esta lógica se implementa con condicionales dentro del nodo `path_generator` de la siguiente manera:

```
if (self.mode == 0):
    for i in range(4):
        x = float(input('Enter x coordinate: '))
        y = float(input('Enter y coordinate: '))
        linear_speed = float(input('Enter linear speed: '))
        while (linear_speed <= 0.005 or linear_speed > 0.3):
            self.get_logger().warning('Linear speed must be in the range (0.005, 0.3]. Please enter a valid value.')
            linear_speed = float(input('Enter linear speed: '))
        angular_speed = float(input('Enter angular speed: '))
        while (angular_speed <= 0.005 or angular_speed > 0.72):
            self.get_logger().warning('Angular speed must be in the range (0.005, 0.72]. Please enter a valid value.')
            angular_speed = float(input('Enter angular speed: '))
```

En el fragmento anterior de código puede observarse también que se implementó una validación de rangos para las velocidades. En el caso de la velocidad lineal, el valor se restringió al intervalo `[0.005, 0.3 m/s]`, mientras que la velocidad angular se acotó a `[0.005, 0.72] rad/s`. Todas estas restricciones se implementaron basándose en la capacidad física del

motor que se observó en el reto semanal 1, es decir el límite inferior es la zona muerta del motor, ya que valores más pequeños a 0.005 no producen movimiento en el robot. Por otro lado, los límites superiores de las velocidades representan el valor máximo en el que el robot responde de forma adecuada, puesto que por encima de ellos el robot alcanza un pico de voltaje y se resetea. Estos límites permitieron aumentar la robustez del sistema al evitar entradas inválidas para este Puzzlebot.

Una vez recibida la información del usuario, el nodo controller se encarga de procesar la información. Primero se suscribe al tópico /pose para recibir el mensaje MakbetsPose y crea un publicador de tipo Twist hacia el tópico /cmd_vel. Después de recibir el mensaje, el controlador calcula la distancia entre puntos consecutivos y el ángulo que el robot debe girar antes de desplazarse hacia cada punto nuevo. Para ello toma como referencia inicial el origen (0,0). Como se mencionó anteriormente, la distancia entre cada punto se calculó a partir de las diferencias de las coordenadas, utilizando la expresión $\sqrt{(dx)^2 + (dy)^2}$. Por otra parte, el ángulo hacia el siguiente punto se calculó mediante la ecuación $\text{atan}(dy, dx)$, la cual permite identificar correctamente la orientación en el plano considerando el cuadrante correspondiente. Asimismo, fue necesario calcular el ángulo relativo de giro restando el ángulo previo al nuevo ángulo deseado. Finalmente, este valor se normalizó en el intervalo $[-\pi, \pi]$ para asegurar que el robot realizará el giro mínimo necesario. Esta lógica se implementa directamente en el controlador y el cálculo de distancias y ángulos se implementó en el método pose_callback, donde también se almacenan estos valores para su posterior uso en la máquina de estados. A continuación se muestra el código para realizar lo anteriormente mencionado:

```
dx = msg.x[i] - prev_x
dy = msg.y[i] - prev_y
self.distances.append(np.sqrt(dx**2 + dy**2))

abs_angle = np.arctan2(dy, dx)
delta = abs_angle - prev_angle
delta = (delta + np.pi) % (2 * np.pi) - np.pi
self.angles.append(delta)
```

Una vez calculadas las distancias y los ángulos el controlador obtiene los tiempos o velocidades requeridas según el modo de operación seleccionado por el usuario. Cuando el sistema opera en modo 0, el usuario ya proporciona las velocidades y el controlador calcula

los tiempos necesarios para realizar el recorrido. Mientras que cuando el sistema opera en modo 1, el usuario proporciona el tiempo y el controlador calcula las velocidades lineales y angulares necesarias para cumplir con ese tiempo. Si alguna de estas velocidades supera los límites permitidos, se aplica una saturación para mantener el movimiento dentro del rango admisible del Puzzlebot. Esto permite que el sistema determinara automáticamente si una trayectoria era alcanzable o no bajo las restricciones dinámicas del robot. Así, la saturación no solo funciona como una protección del sistema, sino también como una manera de manejar trayectorias que exigen velocidades mayores a las físicamente posibles.

Es por ello que en esta parte 2 se implementó una máquina de estados con 5 estados: rotar hacia el punto, pausa, avanzar hacia el punto, pausa y fin de la trayectoria. En el estado de rotación el robot gira sobre su propio eje con velocidad angular constante en la dirección definida por el signo del ángulo calculado. Una vez transcurrido el tiempo de giro correspondiente, el sistema pasa a un estado de pausa de 0.5s. Después el robot entra al estado de avanzar donde se desplaza en línea recta con la velocidad lineal constante durante el tiempo calculado. Al finalizar se realiza otra pausa de 0.5s y si aún queda algún punto por recorrer el sistema vuelve al estado de rotación para dirigirse a la siguiente coordenada. Cuando los 4 puntos fueron alcanzados, la máquina de estado entra al estado final, donde las velocidad angular y lineal se vuelven 0 y por ende se detiene completamente. Esta máquina de estados permitió organizar el movimiento del robot de forma secuencial y clara, haciendo un sistema con control de lazo abierto que pudiera seguir una trayectoria. A continuación se muestra un diagrama de la máquina de estados implementada para esta parte 2:

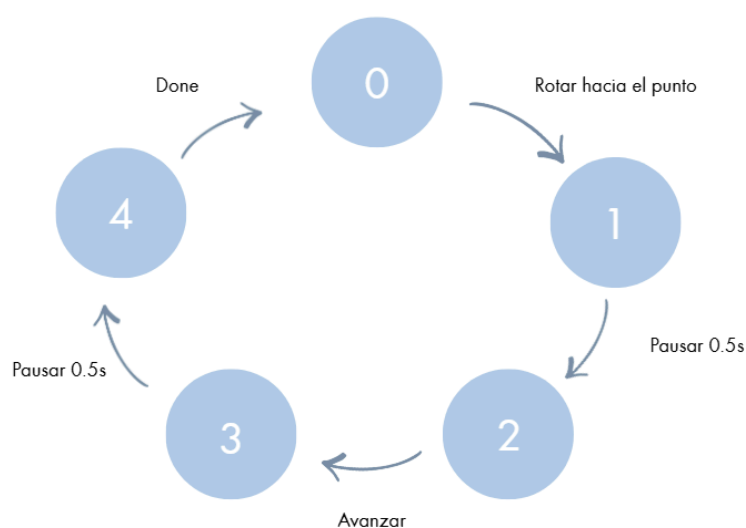


Imagen 4. Diagrama visual de la máquina de estados segunda parte.

Tomando en cuenta todo lo anteriormente mencionado, en este reto semanal la robustez del sistema del controlador se entendió como la capacidad de ejecutar la trayectoria solicitada de manera estable a pesar de las perturbaciones, ruido y no linealidades. Debido a que no se utilizó retroalimentación, la robustez se logró mediante la validación de velocidades de entrada, la identificación práctica de la zona muerta y la saturación, la normalización de los ángulos para realizar giros mínimos, la inclusión de pausas entre movimientos para evitar transiciones abruptas y el añadir valores antes del cálculo de las distancias y ángulos para corregir los valores resultantes. Esto permitió que el sistema se comportará de forma más confiable.

Resultados

Para validar el correcto funcionamiento del sistema, se realizaron pruebas con ambos modos de operación utilizando un archivo de configuración `params.yaml` que permite definir los puntos de la trayectoria, las velocidades y los tiempos sin necesidad de ingresar datos manualmente por terminal. Esto representa una mejora en la usabilidad del sistema, ya que el usuario puede modificar la trayectoria simplemente editando el archivo YAML y relanzando el nodo sin necesidad de recompilar el paquete. El archivo de parámetros utilizado para las pruebas contiene la siguiente configuración:

```
1  ros__parameters:
2    mode: 1  # 0: coordinates + speeds, 1: coordinates + time
3
4    # Coordinates (from mode 0 run)
5    points_x: [2.0, 0.7, 2.0, 0.5]
6    points_y: [1.0, 1.4, -0.8, -1.0]
7
8    # Mode 0: speeds
9    linear_speeds: [0.3, 0.15, 0.3, 0.3]
10   angular_speeds: [0.6, 0.4, 0.3, 0.7]
11
12   # Mode 1: times (from mode 1 run)
13   path_times: [1.0, 5.0, 8.0, 8.0]
```

Imagen 5. Archivo de parámetros de configuración.

Prueba en Modo 0: Coordenadas con velocidades

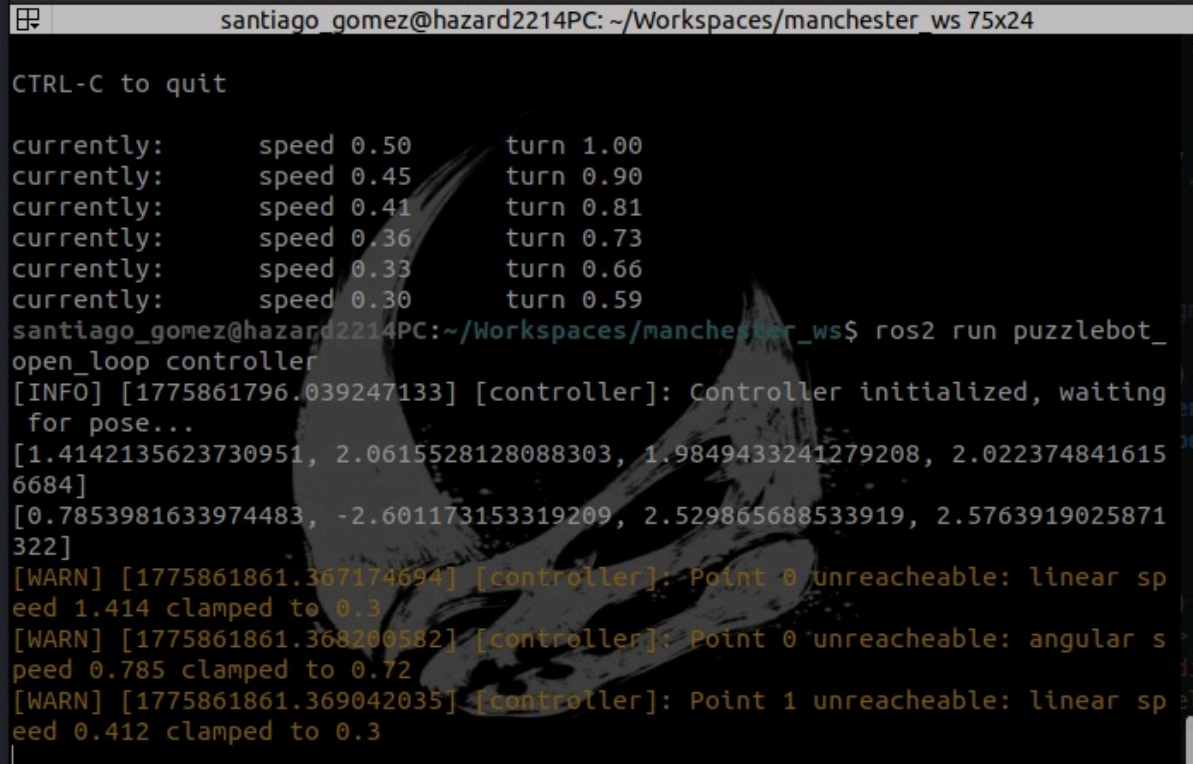
En esta prueba se ejecutó el sistema en modo 0, donde el usuario define las coordenadas de destino junto con las velocidades lineal y angular para cada segmento de la trayectoria. Los puntos utilizados fueron (2, 1), (0.7, 1.4), (2, -0.8) y (0.5, -1), con

velocidades lineales de 0.3, 0.15, 0.3 y 0.3 m/s respectivamente, y velocidades angulares de 0.6, 0.4, 0.3 y 0.7 rad/s. Es importante destacar que la trayectoria incluye coordenadas negativas en el eje Y, lo cual demuestra que el sistema es capaz de operar correctamente en los cuatro cuadrantes del plano cartesiano gracias al cálculo del ángulo mediante $\text{atan2}(dy, dx)$ y la normalización del ángulo relativo en el intervalo $[-\pi, \pi]$.

El controlador calculó correctamente las distancias y los ángulos relativos entre puntos consecutivos, y el robot ejecutó la trayectoria completa siguiendo la secuencia de la máquina de estados: rotar hacia el punto objetivo, pausar 0.5 s, avanzar en línea recta, pausar 0.5 s y continuar con el siguiente punto. El robot completó los cuatro segmentos y se detuvo al finalizar la trayectoria. **Video modo 0:** <https://youtu.be/UaOCp522qXM>

Prueba en Modo 1: Coordenadas con tiempo

Para la prueba en modo 1, se utilizaron las mismas coordenadas pero se definieron tiempos de llegada de 1, 5, 8 y 8 segundos para cada punto respectivamente. En este modo, el controlador calcula automáticamente las velocidades lineales y angulares necesarias para alcanzar cada punto en el tiempo especificado.

A terminal window titled 'santiago_gomez@hazard2214PC: ~/Workspaces/manchester_ws 75x24'. The prompt is 'CTRL-C to quit'. It shows a table of speed and turn values, followed by the command 'ros2 run puzzlebot_open_loop_controller'. The output includes an INFO message about controller initialization, two sets of 4D coordinate arrays, and three WARN messages indicating that linear speeds for Point 0 and Point 1 were clamped to 0.3 m/s due to being unreachable.

```
santiago_gomez@hazard2214PC: ~/Workspaces/manchester_ws 75x24
CTRL-C to quit

currently:      speed 0.50      turn 1.00
currently:      speed 0.45      turn 0.90
currently:      speed 0.41      turn 0.81
currently:      speed 0.36      turn 0.73
currently:      speed 0.33      turn 0.66
currently:      speed 0.30      turn 0.59
santiago_gomez@hazard2214PC:~/Workspaces/manchester_ws$ ros2 run puzzlebot_
open_loop_controller
[INFO] [1775861796.039247133] [controller]: Controller initialized, waiting
for pose...
[1.4142135623730951, 2.0615528128088303, 1.9849433241279208, 2.022374841615
6684]
[0.7853981633974483, -2.601173153319209, 2.529865688533919, 2.5763919025871
322]
[WARN] [1775861861.367174694] [controller]: Point 0 unreachable: linear sp
eed 1.414 clamped to 0.3
[WARN] [1775861861.368200582] [controller]: Point 0 unreachable: angular s
peed 0.785 clamped to 0.72
[WARN] [1775861861.369042035] [controller]: Point 1 unreachable: linear sp
eed 0.412 clamped to 0.3
```

Imagen 6. Se observa un warning cuando un punto es inalcanzable físicamente.

Como se observa en la Imagen 5, al ejecutar esta prueba el controlador generó advertencias de tipo [WARN] indicando que las velocidades calculadas para algunos segmentos excedían los límites físicos del Puzzlebot. Por ejemplo, para el punto 0 la velocidad lineal calculada fue de 1.414 m/s y la velocidad angular de 0.785 rad/s, valores que superan los máximos permitidos de 0.3 m/s y 0.72 rad/s respectivamente. De igual manera, para el punto 1 se calculó una velocidad lineal de 0.412 m/s, la cual también fue saturada al valor máximo. Esto se debe a que el primer punto tiene asignado un tiempo de solo 1 segundo para recorrer una distancia de aproximadamente 2.24 metros, lo cual resulta físicamente inalcanzable para el robot.

Ante esta situación, el sistema no se detuvo ni generó un error, sino que aplicó una saturación automática (clamping) fijando las velocidades a los valores máximos aceptables. Esto permitió que el robot ejecutara la trayectoria de la mejor manera posible dentro de sus capacidades físicas, aunque el tiempo real de ejecución de dichos segmentos fue mayor al solicitado por el usuario.

Video modo 1: <https://youtu.be/yKq1blMLVlg>

Mecanismos de robustez implementados

A pesar de tratarse de un sistema de control en lazo abierto sin retroalimentación de sensores, se implementaron diversos mecanismos para hacer el sistema más robusto ante entradas incorrectas, perturbaciones y limitaciones físicas del robot:

- Validación de rangos de velocidad: En el modo 0, el nodo *path_generator* verifica que las velocidades lineales estén dentro del rango (0.005, 0.3] m/s y las velocidades angulares dentro de (0.005, 0.72] rad/s. Los límites inferiores corresponden a la zona muerta del motor identificada experimentalmente en el reto semanal 1, por debajo de la cual el robot no produce movimiento. Los límites superiores representan los valores máximos antes de que el robot alcance un pico de voltaje y se reinicie.
- Saturación automática de velocidades (clamping): En el modo 1, cuando las velocidades calculadas a partir del tiempo proporcionado superan los límites físicos del robot, el controlador las satura automáticamente al valor máximo permitido y genera una advertencia en la terminal indicando la velocidad original y el valor al que fue fijada. Esto evita que el robot reciba comandos que podrían dañar los motores o causar un reinicio.

- Normalización de ángulos relativos: El sistema calcula el ángulo de giro como la diferencia entre el ángulo absoluto hacia el nuevo punto y el ángulo de orientación actual del robot. Este valor se normaliza al intervalo $[-\pi, \pi]$ mediante la expresión $(\text{delta} + \pi) \% (2\pi) - \pi$, lo que garantiza que el robot siempre realice el giro mínimo necesario, evitando rotaciones innecesarias de más de 180° .
- Soporte para coordenadas negativas: El uso de `np.arctan2(dy, dx)` permite calcular correctamente la orientación hacia puntos en cualquier cuadrante del plano, incluyendo coordenadas negativas tanto en X como en Y, como se demostró en las pruebas realizadas.
- Pausas entre movimientos: Se incluyeron pausas de 0.5 segundos entre cada transición de estado (después de rotar y después de avanzar), lo cual permite que la inercia del robot se disipe antes de iniciar el siguiente movimiento, reduciendo el efecto de transiciones abruptas en la precisión de la trayectoria.
- Factores de calibración experimental: Como se documentó anteriormente, se aplicaron factores de corrección de 0.95 para la distancia y 0.895 para el ángulo, obtenidos mediante un proceso iterativo de calibración que compensan las no linealidades de los motores y la fricción del entorno.
- Configuración mediante archivo YAML: La migración de la entrada de datos desde la terminal hacia un archivo `params.yaml` cargado mediante un launch file de ROS 2 permite una ejecución repetible y reduce la posibilidad de errores humanos al ingresar valores manualmente.

Conclusiones

El sistema de control en lazo abierto implementado demostró ser funcional para la generación y seguimiento de trayectorias, aunque su precisión depende directamente de la calibración inicial y de las condiciones del entorno. La ausencia de retroalimentación hace que cualquier perturbación externa, como variaciones en la fricción del piso o irregularidades en la superficie, se acumule a lo largo de la trayectoria sin posibilidad de corrección, lo cual representa la principal limitación de este enfoque.

El proceso de calibración experimental fue fundamental para lograr un desempeño aceptable del robot. Sin los factores de corrección aplicados a la distancia y al ángulo, el robot presentaba sobrepasos significativos, recorriendo 2.2 metros en lugar de 2 y girando 110° en

lugar de 90°. Esto evidencia que las características físicas del robot, como la inercia de los motores y el tiempo de respuesta del sistema, no pueden ignorarse al momento de diseñar un controlador, incluso en un esquema de lazo abierto.

La implementación de mecanismos de robustez como la validación de rangos de velocidad, la saturación automática y la normalización de ángulos permitió que el sistema operara de forma estable aun cuando el usuario proporcionara entradas que excedieran las capacidades físicas del Puzzlebot. En particular, la saturación de velocidades en el modo 1 demostró ser una estrategia efectiva para evitar que el robot recibiera comandos que pudieran dañar los motores o causar un reinicio del sistema, sacrificando la precisión temporal a cambio de la integridad del hardware.

Finalmente, este reto permitió comprender las limitaciones inherentes a los sistemas de control en lazo abierto y la importancia de la calibración y la validación de entradas para lograr un comportamiento confiable. Estas bases serán fundamentales para el desarrollo de controladores en lazo cerrado en futuros retos, donde la retroalimentación de sensores permitirá corregir errores de manera dinámica y alcanzar una mayor precisión en el seguimiento de trayectorias.

Referencias

1. ROS 2. (2020). Documentación de ROS Mensaje de Pose.
http://docs.ros2.org/foxy/api/geometry_msgs/msg/Pose.html
2. ROS 2. (2025). Documentación de ROS Mensaje Twist.
https://docs.ros.org/en/noetic/api/geometry_msgs/html/msg/Twist.html
3. Christ, R & Wernili, R. (2014). The ROV Manual, A User Guide for Remotely Operated Vehicles.
<https://www.sciencedirect.com/topics/computer-science/open-loop-control>